

Sección 6: Implementar límites de protección y responsabilidad (10-15%)

Esta sección evalúa la capacidad de diseñar e implementar controles de protección (guardrails) y marcos de responsabilidad para agentes de IA en el ecosistema GitHub. Cubre la clasificación de riesgos, la definición de niveles de autonomía, la implementación de barandillas de seguridad y la integración de flujos de trabajo con supervisión humana (human-in-the-loop), todo ello equilibrando velocidad de entrega con seguridad y cumplimiento normativo.

6.1 Definición de niveles de autonomía

Conceptos clave

Clasificación de acciones por riesgo

Cada acción que un agente puede realizar debe clasificarse según tres dimensiones de riesgo:

Dimensión	Descripción	Ejemplos
Riesgo operativo	Impacto en la estabilidad y disponibilidad de sistemas	Despliegues a producción, cambios en infraestructura, modificaciones de pipelines CI/CD
Riesgo de seguridad	Exposición a vulnerabilidades o accesos no autorizados	Cambios en permisos, gestión de secretos, configuraciones de red, modificación de políticas de acceso
Riesgo de cumplimiento	Violación de normativas, políticas internas o estándares regulatorios	Modificación de datos personales, cambios en políticas de retención, alteración de registros de auditoría

Niveles de riesgo en GitHub

El examen espera que clasifiques acciones concretas en estos niveles:

Nivel	Descripción	Ejemplos de acciones	Intervención humana
Bajo	Acciones de solo lectura y análisis	Analizar código, generar reportes, revisar logs, ejecutar linters, escanear dependencias	Ninguna o mínima
Medio	Cambios de código con revisión	Crear PRs, modificar código fuente, actualizar documentación, agregar tests	Revisión de PR requerida

Nivel	Descripción	Ejemplos de acciones	Intervención humana
Alto	Despliegues y cambios de infraestructura	Deploy a producción, cambios en IaC (Terraform/Bicep), modificación de datos en bases de datos, cambios en configuración de servicios	Aprobación explícita + revisión
Crítico	Configuraciones de seguridad y acceso	Cambios en permisos de repositorio, rotación de secretos, modificación de branch protection rules, cambios en RBAC, alteración de políticas de organización	Múltiples aprobaciones + auditoría

Los cuatro niveles de autonomía del agente

NIVELES DE AUTONOMÍA	
TOTALMENTE AUTÓNOMO	El agente ejecuta y completa la acción sin intervención. Ejemplo: auto-merge de PRs que pasan todos los checks en ramas no protegidas. → Riesgo BAJO
SEMI-AUTÓNOMO	El agente ejecuta la acción pero requiere validación humana antes de que surta efecto. Ejemplo: crear PR y esperar revisión/aprobación. → Riesgo MEDIO
SUPERVISADO	El agente genera un plan o propuesta, pero el humano ejecuta manualmente. Ejemplo: plan de migración que humano revisa y aplica paso a paso. → Riesgo ALTO
RESTRINGIDO (Solo lectura)	El agente solo puede leer y analizar. No tiene capacidad de escritura ni ejecución. Ejemplo: auditoría de seguridad, análisis de código sin modificaciones. → Cualquier nivel de riesgo del recurso

Matriz de decisión: velocidad vs. seguridad

El objetivo es **maximizar la velocidad de entrega** sin comprometer la seguridad ni los estándares de IA responsable. No toda acción requiere aprobación humana — solo aquellas donde el riesgo lo justifica.

Velocidad de entrega ← → Control y seguridad

Autónomo	Semi-autónomo	Supervisado	Restringido
●	●	●	●
Máxima velocidad	Alta velocidad	Moderada velocidad	Mínima velocidad
Mínimo control	Moderado control	Alto control	Máximo control

Principio clave para el examen: La asignación de niveles de autonomía debe ser proporcional al riesgo. Agregar aprobaciones innecesarias reduce la velocidad sin mejorar la seguridad (*approval fatigue*).

Aplicación práctica en GitHub

1. Configuración de autonomía con GitHub Copilot Coding Agent

El agente Copilot opera predominantemente en modo **semi-autónomo**: crea un PR con los cambios propuestos y espera revisión humana. Esto se configura mediante:

- **Branch protection rules:** Exigir revisiones antes de merge
- **CODEOWNERS:** Asignar revisores automáticos según las rutas de archivos modificados
- **Required status checks:** Garantizar que CI/CD pasa antes de permitir merge

```
# Ejemplo de CODEOWNERS para control de revisión
# Cambios en infraestructura requieren aprobación del equipo de plataforma
/infraestructura/    @org/platform-team
/terraform/          @org/platform-team @org/security-team

# Cambios en código de aplicación requieren aprobación del equipo de desarrollo
/src/                @org/dev-team

# Cambios en configuración de seguridad requieren aprobación del equipo de
seguridad
/.github/workflows/ @org/security-team
/security/           @org/security-team @org/compliance-team
```

2. Rulesets para control granular

Los **Repository Rulesets** permiten definir reglas a nivel de organización o repositorio:

```
Ruleset: "agente-produccion"
├─ Target: ramas que coinciden con "main", "release/*"
├─ Reglas:
│   └─ Require pull request before merging
│       └─ Required approvals: 2
│       └─ Dismiss stale reviews: true
```

```

└─ Require review from CODEOWNERS: true
└─ Require status checks to pass
    └─ CI build
    └─ Security scan
    └─ Compliance check
└─ Require signed commits
└─ Block force pushes
└─ Bypass: ninguno (ni siquiera administradores)

```

3. Environment protection rules para despliegues

```

Entorno: produccion
└─ Required reviewers: @org/release-managers
└─ Wait timer: 15 minutos
└─ Deployment branches: solo "main"
└─ Custom protection rules:
    └─ Verificación de compliance externa

```

4. Clasificación práctica de acciones del agente

Acción del agente	Nivel de riesgo	Autonomía recomendada	Control de GitHub
Analizar código y sugerir mejoras	Bajo	Autónomo/Restringido	Ninguno especial
Crear PR con corrección de bug	Medio	Semi-autónomo	Branch protection + required reviews
Modificar workflow de CI/CD	Alto	Supervisado	CODEOWNERS + múltiples aprobaciones
Cambiar configuración de secretos	Crítico	Restringido	Solo lectura + notificación
Auto-merge de dependabot updates	Bajo-Medio	Autónomo (con checks)	Required status checks
Deploy a staging	Medio	Semi-autónomo	Environment protection
Deploy a producción	Alto	Supervisado	Environment protection + wait timer + reviewers
Modificar permisos de repositorio	Crítico	Restringido	Solo vía administrador humano

Puntos clave para el examen

1. **Los cuatro niveles de autonomía** (totalmente autónomo, semi-autónomo, supervisado, restringido) y cuándo aplicar cada uno.
2. **Las tres dimensiones de riesgo** (operativo, seguridad, cumplimiento) son independientes — una acción puede ser de bajo riesgo operativo pero alto riesgo de cumplimiento.
3. **La clasificación de riesgo determina el nivel de autonomía**, no al revés.
4. **El modo semi-autónomo (crear PR + esperar revisión) es el patrón más común** para agentes en GitHub y el equilibrio más frecuente entre velocidad y seguridad.
5. **Approval fatigue**: agregar demasiadas aprobaciones no mejora la seguridad; la reduce porque los revisores dejan de prestar atención. Solo puertas de aprobación proporcionales al riesgo.
6. **Los rulesets de organización** pueden imponer niveles de autonomía consistentes en todos los repositorios.
7. **El agente Copilot siempre crea un PR** — nunca hace push directamente a ramas protegidas. Esto es un control arquitectónico de semi-autonomía.

6.2 Implementación de barandillas de seguridad y flujos de trabajo con humanos en el circuito

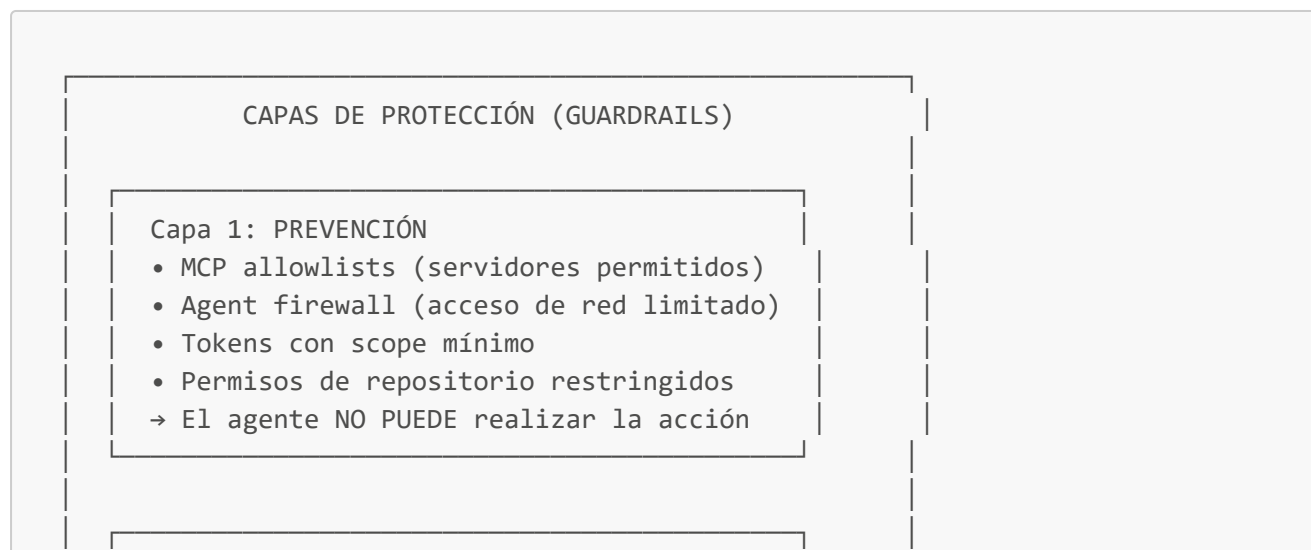
Conceptos clave

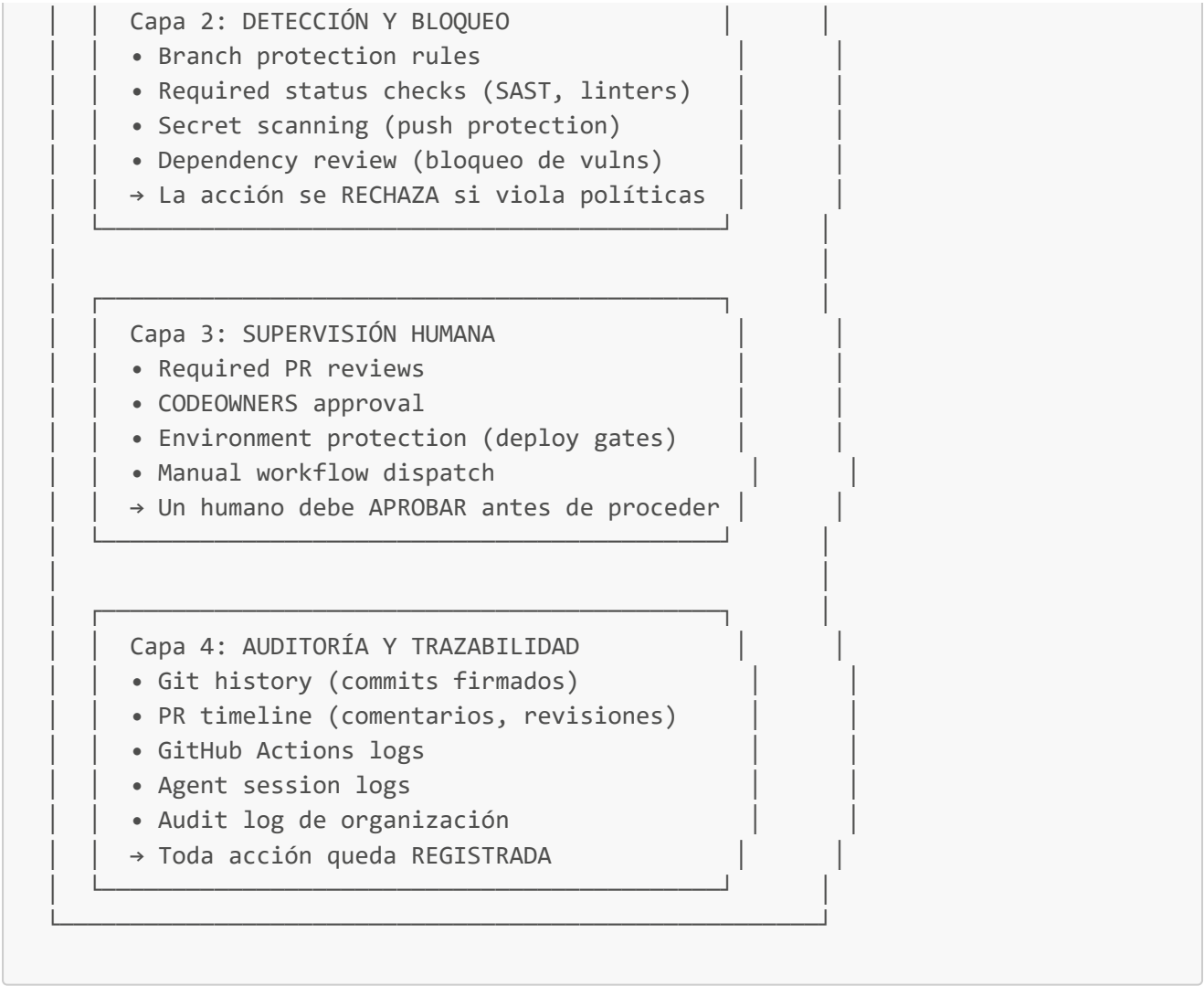
Identificación de acciones que requieren juicio humano

No toda decisión puede (ni debe) automatizarse. Las acciones que requieren juicio humano incluyen:

- **Decisiones con contexto de negocio**: Priorización de features, trade-offs arquitectónicos, decisiones de UX
- **Acciones irreversibles**: Eliminación de datos, cambios en producción que no se pueden revertir fácilmente
- **Cambios sensibles al cumplimiento**: Modificaciones que afectan datos regulados (PII, PHI, PCI)
- **Decisiones éticas**: Contenido que podría ser dañino, sesgado o discriminatorio
- **Cambios de alto impacto en seguridad**: Modificaciones de permisos, configuración de autenticación, políticas de red

Bloqueo de acciones que infringen directivas





Principio de privilegio mínimo (Least Privilege)

El agente debe tener **solo los permisos estrictamente necesarios** para realizar su tarea:

Control	Implementación	Propósito
Tokens con scope mínimo	<code>repo:read</code> , <code>pull_request:write</code> en lugar de <code>repo:admin</code>	Limitar qué APIs puede llamar el agente
Permisos de repositorio	Acceso solo a repos específicos, no a toda la organización	Limitar el radio de impacto
MCP allowlists	Lista explícita de servidores MCP permitidos a nivel de organización	Controlar qué herramientas externas puede usar el agente
Agent firewall	Configurar qué dominios/IPs puede contactar el agente en la nube	Prevenir exfiltración de datos o contacto con servicios no autorizados
Contextos de ejecución limitados	Runners efímeros, contenedores aislados, sin acceso a red interna	Minimizar la superficie de ataque

Control	Implementación	Propósito
Acceso temporal	Tokens de corta duración, permisos que expiran	Reducir ventana de exposición

MCP Allowlists (control de servidores MCP)

Organización: mi-empresa

MCP Servers permitidos (allowlist):

github/github-mcp-server

empresainternal-tools-mcp

terceranalytictics-mcp

✓ Permitido

✓ Permitido

✓ Permitido (con restricciones)

MCP Servers bloqueados:

desconocido/random-mcp

atacante/malicious-mcp

✗ No en allowlist

✗ No en allowlist

Política: Solo servidores en la allowlist pueden ser invocados por agentes en esta organización

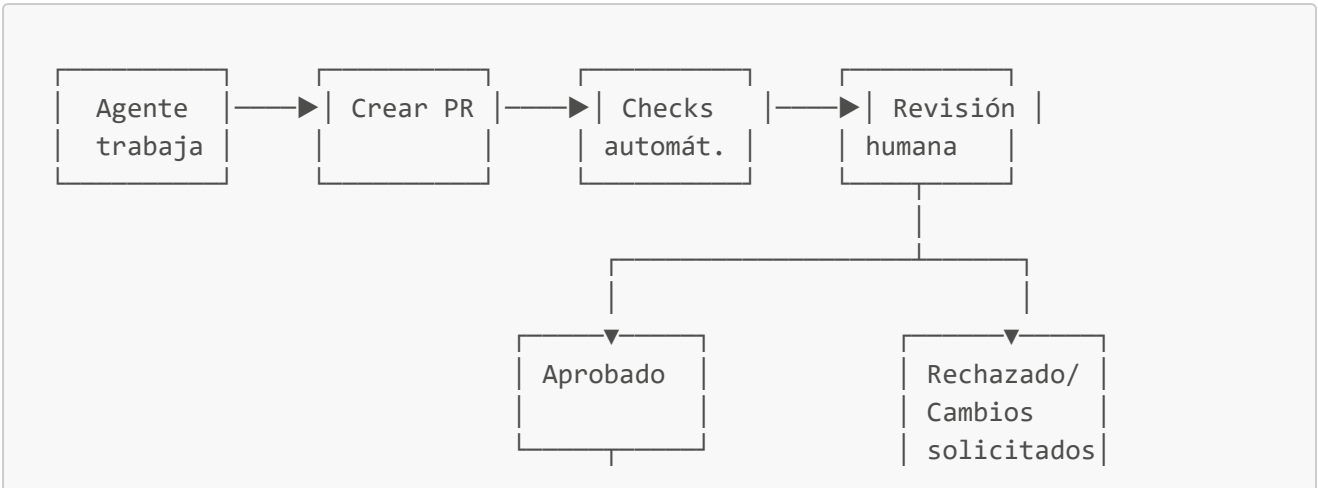
Concepto clave: Las MCP allowlists son un control **preventivo** a nivel de organización. Si un servidor MCP no está en la lista, el agente simplemente no puede usarlo, independientemente de lo que el usuario solicite.

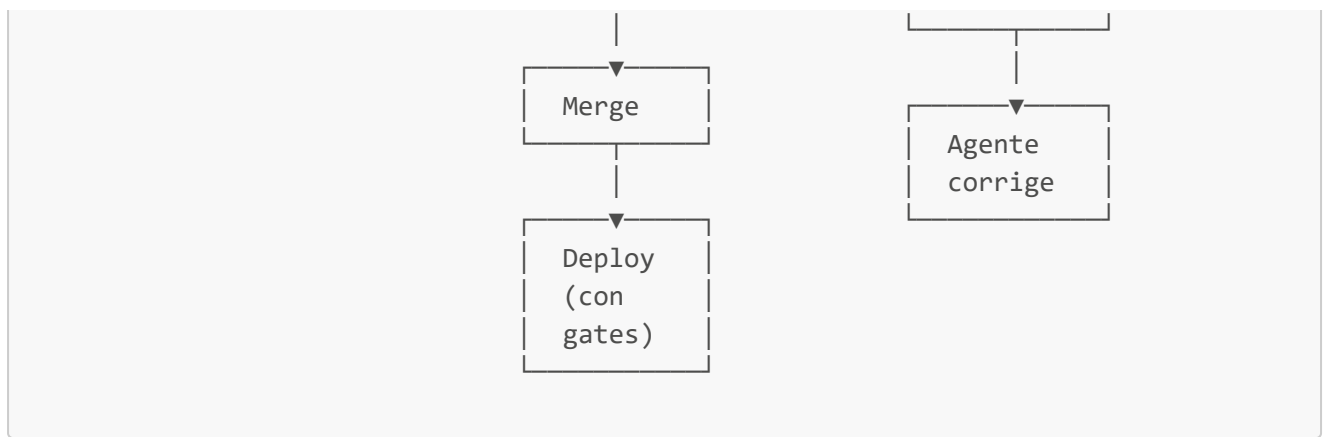
Agent Firewall (control de acceso de red)

El firewall del agente controla las conexiones de red que el agente en la nube puede realizar:

- **Dominios permitidos:** Solo puede contactar servicios aprobados
- **Bloqueo por defecto:** Todo tráfico no explícitamente permitido se rechaza
- **Prevención de exfiltración:** Impide que código malicioso en el repo envíe datos a servidores externos
- **Logging:** Todas las conexiones intentadas quedan registradas

Flujos de trabajo Human-in-the-Loop en GitHub





Mecanismos de human-in-the-loop en GitHub:

1. PR Reviews como puertas de aprobación

- El agente crea un PR → humano revisa → aprueba o solicita cambios
- Configurable: número mínimo de aprobaciones, revisores específicos (CODEOWNERS)
- El PR actúa como registro de auditoría completo

2. Required Status Checks

- Tests automatizados, SAST, linting deben pasar antes de permitir merge
- Combinan validación automática con supervisión humana
- Bloquean merge incluso si el revisor aprueba pero los checks fallan

3. Environment Protection Rules

- Aprobación manual requerida para despliegues a entornos sensibles
- Wait timers que dan tiempo para cancelar despliegues
- Restricción de qué ramas pueden desplegar a qué entornos

4. Manual Workflow Dispatch

- Workflows de GitHub Actions que requieren activación manual
- El agente prepara pero no ejecuta; el humano decide cuándo activar

Rutas controladas para cambios irreversibles

Para cambios que no se pueden revertir fácilmente:

Tipo de cambio	Ruta controlada	Justificación
Eliminación de datos	Aprobación de 2+ personas + periodo de gracia	Irreversible
Cambio de configuración de seguridad	Aprobación de equipo de seguridad + registro en audit log	Alto impacto
Modificación de permisos de acceso	Solicitud formal + aprobación de propietario del recurso	Riesgo de escalada de privilegios

Tipo de cambio	Ruta controlada	Justificación
Deploy a producción	Aprobación de release manager + verificación post-deploy	Impacto en usuarios finales
Cambio de políticas de organización	Aprobación de múltiples administradores	Impacto organizacional amplio
Rotación de secretos/credenciales	Proceso automatizado con verificación + rollback plan	Puede romper integraciones

Equilibrio entre velocidad y seguridad

Principio fundamental: Minimizar las aprobaciones que no reducen materialmente el riesgo.

BUENA PRÁCTICA:

Riesgo bajo → Sin aprobación manual → Ejecución inmediata
 Riesgo medio → 1 aprobación → Ejecución en minutos
 Riesgo alto → 2+ aprobaciones → Ejecución en horas
 Riesgo crit. → Múltiples aprobaciones → Ejecución planificada

MALA PRÁCTICA:

Todo requiere 3 aprobaciones → Approval fatigue →
 Los revisores aprueban sin leer → MAYOR riesgo

Estrategias para mantener velocidad:

- **Auto-merge para bajo riesgo:** Si todos los status checks pasan y el cambio es de bajo riesgo, permitir merge automático
- **Aprobaciones asíncronas:** No bloquear al agente esperando; puede trabajar en otras tareas
- **Fast-track para cambios triviales:** Documentación, typos, actualizaciones de dependencias menores
- **Escalado progresivo:** Empezar con autonomía alta y reducir solo si se detectan problemas

Aplicación práctica en GitHub

1. Configuración completa de guardrails para un agente

```
# .github/copilot-instructions.md (directivas del agente)
# Instrucciones de seguridad para el agente:
# - NO modificar archivos en /security/ o /.github/workflows/
# - NO crear tokens ni gestionar secretos
# - SIEMPRE crear PR, nunca push directo
# - Ejecutar tests antes de marcar como listo para revisión
```

2. Branch Protection + CODEOWNERS (semi-autonomía)

```

Repositorio: mi-aplicacion
├─ Branch: main
│   ├── Require PR before merging: ☒
│   ├── Required approvals: 1
│   ├── Dismiss stale reviews: ☒
│   ├── Require CODEOWNERS review: ☒
│   ├── Required status checks:
│   │   ├── ci/build ☒
│   │   ├── ci/test ☒
│   │   ├── security/codeql ☒
│   │   └── compliance/license-check ☒
│   ├── Require signed commits: ☒
│   └── Block force pushes: ☒
└─ CODEOWNERS:
    ├── * @org/dev-team
    ├── /infrastructure/** @org/platform-team
    ├── /src/auth/** @org/security-team
    └── /.github/** @org/devops-team

```

3. Flujo completo: agente con environment protection

1. Issue asignado al agente Copilot
2. Agente crea branch y hace cambios
3. Agente crea PR → CODEOWNERS auto-asignados como revisores
4. Status checks ejecutan automáticamente:
 - ├─ Build ☒
 - ├─ Tests ☒
 - ├─ CodeQL scan ☒
 - └─ License check ☒
5. Revisor humano aprueba PR
6. PR se mergea a main
7. Workflow de deploy se activa:
 - ├─ Deploy a staging → automático
 - ├─ Tests de integración en staging ☒
 - ├─ Deploy a producción → REQUIERE APROBACIÓN
 - └─ Release manager aprueba
 - └─ Verificación post-deploy ☒

4. Configuración de permisos mínimos para el agente

```

# Permisos del GitHub App / token del agente
permissions:
  contents: read          # Leer código
  pull-requests: write    # Crear y actualizar PRs
  issues: read            # Leer issues asignados

```

```

checks: read          # Ver resultados de checks
# NO incluir:
# contents: write     # No push directo
# administration: write # No cambiar config del repo
# secrets: write      # No gestionar secretos
# members: write      # No cambiar miembros

```

5. Pista de auditoría completa

Cada acción del agente deja rastro en múltiples niveles:

Fuente de auditoría	Qué registra	Retención
Git history	Commits del agente (autor identificable)	Permanente
PR timeline	Comentarios, revisiones, aprobaciones, checks	Permanente
GitHub Actions logs	Ejecución de workflows, resultados de checks	90 días (configurable)
Agent session logs	Conversación completa, herramientas usadas, decisiones	Según política
Audit log de organización	Eventos administrativos, cambios de permisos	90-180 días (según plan)

6. IA Responsable en el contexto de agentes

- **Seguridad de contenido:** El agente no debe generar código malicioso, contenido dañino o sesgado
- **Transparencia:** Los cambios del agente deben ser claramente identificables (commits firmados, autor "copilot")
- **Evitar salidas dañinas:** Filtros de contenido y validación de que el código generado no introduce vulnerabilidades
- **Supervisión continua:** Monitoreo de patrones de comportamiento anómalos del agente
- **Trazabilidad:** Poder reconstruir el razonamiento del agente para cualquier cambio

Puntos clave para el examen

1. **Las cuatro capas de protección** (prevención, detección/bloqueo, supervisión humana, auditoría) y qué controles de GitHub pertenecen a cada una.
2. **MCP allowlists** son controles **preventivos a nivel de organización** — si un servidor MCP no está en la lista, el agente no puede usarlo.
3. **Agent firewall** controla el acceso de red del agente en la nube, previniendo exfiltración de datos y contacto con servicios no autorizados.
4. **Privilegio mínimo:** tokens con scope limitado, permisos de repositorio restringidos, contextos de ejecución aislados. El agente solo debe poder hacer lo estrictamente necesario.
5. **PR como mecanismo principal de human-in-the-loop:** el PR es simultáneamente mecanismo de revisión, puerta de aprobación y registro de auditoría.

6. **Environment protection rules** son el mecanismo para controlar despliegues: reviewers requeridos, wait timers, restricción de ramas.
 7. **No toda acción necesita aprobación humana** — solo puertas proporcionales al riesgo. El exceso de aprobaciones causa *approval fatigue* y paradójicamente reduce la seguridad.
 8. **Cambios irreversibles o sensibles al cumplimiento** requieren rutas controladas: múltiples aprobaciones, periodos de gracia, planes de rollback.
 9. **La pista de auditoría** es multi-capa: git history + PR timeline + Actions logs + session logs + org audit log.
 10. **IA responsable** incluye seguridad de contenido, transparencia (identificar cambios del agente), y trazabilidad del razonamiento.
 11. **Rulesets vs Branch Protection**: los rulesets permiten aplicar reglas consistentes a nivel de organización; branch protection es por repositorio.
 12. **El equilibrio velocidad-seguridad** es un tema central: el examen evaluará si sabes aplicar controles proporcionales al riesgo sin sobre-ingeniería.
-

Recursos

- [Documentación de GitHub: Branch protection rules](#)
- [Documentación de GitHub: Repository rulesets](#)
- [Documentación de GitHub: CODEOWNERS](#)
- [Documentación de GitHub: Environment protection rules](#)
- [Documentación de GitHub: Copilot coding agent](#)
- [Documentación de GitHub: Managing MCP servers for Copilot](#)
- [Documentación de GitHub: Agent firewall](#)
- [Documentación de GitHub: Audit log](#)
- [GitHub: Responsible use of Copilot](#)
- [Principio de privilegio mínimo \(NIST\)](#)